

MANUAL DE GITHUB DE NOVATO A AVANZADO

Guía exhaustiva para desarrolladores en Microsoft Windows con PowerShell 7+, Git Credential Manager y GitHub CLI

[CLI & Web Workflows](#)[SSH & GPG Signing](#)[GitHub Actions CI/CD](#)[Issues & Projects v2](#)[Security & Dependabot](#)[Packages & Releases](#)[API & Scripting](#)

Manual de GitHub: De Novato a Avanzado en PowerShell (Windows)

Plataforma: Microsoft Windows 10 / Windows 11 / Windows Server

Shell: PowerShell 7+ (Core) / Windows PowerShell 5.1 (Windows Terminal)

Herramientas: Git for Windows 2.40+, GitHub CLI (`gh`), Git Credential Manager (GCM), OpenSSH for Windows, Posh-Git, Git LFS

Índice de Contenidos

1. [Parte I: Fundamentos y Configuración del Entorno en Windows](#)
 - 1.1 [Diferencias entre Git y GitHub](#)
 - 1.2 [Instalación con Winget y Optimización de PowerShell](#)
 - 1.3 [Configuración de Identidad y Manejo de CRLF / Rutas Largas](#)
 - 1.4 [Autenticación con Git Credential Manager y GitHub CLI](#)
 - 1.5 [Configuración de OpenSSH en Windows](#)
 - 1.6 [Firma Criptográfica de Commits en Windows \(GPG y SSH\)](#)
2. [Parte II: Flujo de Trabajo Esencial \(Nivel Novato\)](#)
 - 2.1 [Creación y Clonación de Repositorios desde PowerShell](#)
 - 2.2 [El Ciclo de Trabajo en Windows: Working Tree, Index y Commit](#)
 - 2.3 [Staging Selectivo y Estándar Conventional Commits](#)
 - 2.4 [Sincronización con GitHub](#)
 - 2.5 [Control de Archivos Ignorados en Windows](#)
3. [Parte III: Ramas, Fusiones y Estrategias Colaborativas \(Nivel Intermedio\)](#)
 - 3.1 [Gestión de Ramas con PowerShell y Posh-Git](#)
 - 3.2 [Los Tres Métodos de Merge en GitHub](#)
 - 3.3 [Modelos de Flujo: GitHub Flow, Git Flow y Forking](#)
 - 3.4 [Operaciones Avanzadas: Stash, Cherry-Pick y Rebase Interactivo](#)
 - 3.5 [El Salvavidas: Recuperación con Reflog](#)
4. [Parte IV: Soluciones por Temas a la Edición Concurrente del Mismo Archivo](#)
 - 4.1 [Tema 1: Prevención y Buenas Prácticas de Equipo](#)
 - 4.2 [Tema 2: Fusión Automática en Distintas Líneas](#)
 - 4.3 [Tema 3: Conflicto Directo de Fusión con VS Code](#)
 - 4.4 [Tema 4: Elección Total de Versión](#)

- 4.5 [Tema 5: Cambios Locales sin Confirmar al hacer Pull](#)
 - 4.6 [Tema 6: Push Rechazado por Desfase y Rebase Seguro](#)
 - 4.7 [Tema 7: Resolución de Conflictos en Pull Requests](#)
 - 4.8 [Tema 8: Conflicto de Modificación vs Eliminación](#)
 - 4.9 [Tema 9: Archivos Binarios y Bloqueo con Git LFS en Windows](#)
5. [Parte V: Herramientas Modernas de Productividad Avanzada](#)
- 5.1 [Git Worktrees en Windows: Trabajar en Múltiples Ramas sin Conmutar](#)
 - 5.2 [Depuración Binaria con Git Bisect y Auditoría con Blame](#)
 - 5.3 [GitHub Codespaces desde PowerShell](#)
 - 5.4 [GitHub Copilot CLI en PowerShell](#)
 - 5.5 [Git Hooks y Validación con Pre-commit en Windows](#)
6. [Parte VI: Gestión de Proyectos y Ecosistema GitHub](#)
- 6.1 [GitHub Issues, Milestones y Labels desde PowerShell](#)
 - 6.2 [Pull Requests y Revisiones desde GitHub CLI](#)
 - 6.3 [GitHub Projects \(v2\): Automatizaciones y Tableros](#)
 - 6.4 [GitHub Discussions y Wikis en Windows](#)
7. [Parte VII: Automatización y CI/CD con GitHub Actions](#)
- 7.1 [Estructura de Pipelines y Sintaxis YAML](#)
 - 7.2 [Workflows con Runners `windows-latest` y Shell `pwsh`](#)
 - 7.3 [Secretos, Matrices Multiplataforma y Caching](#)
 - 7.4 [Configuración de un Self-Hosted Runner como Servicio de Windows](#)
8. [Parte VIII: Distribución, Paquetes y Publicación](#)
- 8.1 [GitHub Releases: Binarios `.exe`, `.msi` y Archivos `.zip`](#)
 - 8.2 [GitHub Packages: Registro de Paquetes NuGet y npm](#)
 - 8.3 [GitHub Pages: Publicación Automatizada](#)
9. [Parte IX: Seguridad, Gobernanza y Administración](#)
- 9.1 [Reglas de Protección de Ramas y Rulesets](#)
 - 9.2 [Dependabot, Secret Scanning y Push Protection](#)
 - 9.3 [Análisis de Seguridad SAST con CodeQL](#)
 - 9.4 [Gobernanza con CODEOWNERS y Permisos](#)
10. [Parte X: Catálogo Maestro de Incidentes y Soluciones en Windows](#)
- 10.1 [Incidente 1: Fuga Accidental de Credenciales o Tokens](#)
 - 10.2 [Incidente 2: Push Rechazado por Archivo Mayor a 100 MB](#)
 - 10.3 [Incidente 3: Reversión Limpia de un Merge Roto en Producción](#)

- 10.4 [Incidente 4: Rebase de una Rama Compartida](#)
 - 10.5 [Incidente 5: Restaurar una Rama Remota Borrada en GitHub](#)
 - 10.6 [Incidente 6: Corrección Masiva de Correo en Commits Históricos](#)
 - 10.7 [Incidente 7: Ataques de Pwn Request en GitHub Actions](#)
 - 10.8 [Incidente 8: Bucle Infinito de Workflows en GitHub Actions](#)
 - 10.9 [Incidente 9: Optimización de Repositorios Pesados en Windows](#)
 - 10.10 [Incidente 10: Tags o Etiquetas Desincronizadas en Windows](#)
-

Parte I: Fundamentos y Configuración del Entorno en Windows

1.1 Diferencias entre Git y GitHub

- **Git:** Motor de control de versiones distribuido que opera localmente en Windows. Gestiona el historial de archivos mediante instantáneas criptográficas dentro de la carpeta oculta `.git` sin requerir conexión a la red.
- **GitHub:** Plataforma en la nube de desarrollo colaborativo que centraliza repositorios remotos, facilita revisiones de código mediante Pull Requests, integra automatización con GitHub Actions y provee herramientas de seguridad avanzada.

1.2 Instalación con Winget y Optimización de PowerShell

Abre una ventana de **PowerShell como Administrador**:

```
POWERSHELL

winget install --id Git.Git -e --source winget
winget install --id GitHub.cli -e --source winget
Install-Module posh-git -Scope CurrentUser -Force
```

¿Qué hace este comando?

Descarga e instala los paquetes oficiales de 64 bits de Git y GitHub CLI mediante Winget, e instala el módulo `posh-git` para autocompletado y visualización de ramas en el prompt.

```
POWERSHELL

if (!(Test-Path -Path $PROFILE)) { New-Item -ItemType File -Path $PROFILE -Force }
Add-Content -Path $PROFILE -Value "Import-Module posh-git"
& $PROFILE
```

¿Qué hace este comando?

Crea tu script de perfil de PowerShell si no existe, añade la importación de `posh-git` y lo ejecuta en la sesión actual.

1.3 Configuración de Identidad y Manejo de CRLF / Rutas Largas

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu-correo@ejemplo.com"
git config --global core.autocrlf true
git config --global core.longpaths true
git config --global init.defaultBranch main
```

¿Qué hace este comando?

Define la autoría de tus commits, normaliza los saltos de línea Windows CRLF (`\r\n`) a LF al enviar a GitHub, habilita soporte para rutas de más de 260 caracteres en Windows y fija `main` como rama principal.

1.4 Autenticación con Git Credential Manager y GitHub CLI

```
git config --global credential.helper manager
gh auth login --hostname github.com --git-protocol https --web
gh auth status
```

¿Qué hace este comando?

Configura **Git Credential Manager (GCM)** para almacenar credenciales seguras en el Administrador de Credenciales de Windows y valida tu cuenta en GitHub CLI.

1.5 Configuración de OpenSSH en Windows

```
Get-Service ssh-agent | Set-Service -StartupType Automatic
Start-Service ssh-agent
ssh-keygen -t ed25519 -C "tu-correo@ejemplo.com" -f "$HOME\.ssh\id_ed25519"
ssh-add "$HOME\.ssh\id_ed25519"
```

¿Qué hace este comando?

Habilita el servicio de fondo OpenSSH de Windows, genera un par de claves Ed25519 seguras y carga la clave privada en el agente.

```
$sshConfig = @"
Host github.com
    HostName ssh.github.com
    Port 443
    User git
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes
"@
Set-Content -Path "$HOME\.ssh\config" -Value $sshConfig
gh ssh-key add "$HOME\.ssh\id_ed25519.pub" --title "Windows-PowerShell-Workstation"
ssh -T git@github.com
```

¿Qué hace este comando?

Configura el acceso SSH por el puerto 443 para eludir firewalls, sube la clave pública a tu cuenta de GitHub y verifica la conexión.

1.6 Firma Criptográfica de Commits en Windows (GPG y SSH)

```
git config --global gpg.format ssh
git config --global user.signingkey "$HOME/.ssh/id_ed25519.pub"
git config --global commit.gpgsign true
git config --global tag.gpgsign true
```

¿Qué hace este comando?

Firma automáticamente todos los commits y etiquetas con tu clave SSH para certificar tu autoría con la insignia **Verified** en GitHub.

Parte II: Flujo de Trabajo Esencial (Nivel Novato)

2.1 Creación y Clonación de Repositorios desde PowerShell

```
POWERSHELL

New-Item -ItemType Directory -Path ".\mi-proyecto-windows" -Force
Set-Location ".\mi-proyecto-windows"
git init
Set-Content -Path "README.md" -Value "# Proyecto Windows en GitHub"
git add README.md
git commit -m "docs: inicializar repositorio"
gh repo create mi-proyecto-windows --public --source=. --remote=origin --push
```

¿Qué hace este comando?

Crea la carpeta, inicia Git, genera el primer commit y publica el repositorio en GitHub vinculando el remoto en una sola operación.

```
POWERSHELL

git clone git@github.com:usuario/mi-repositorio.git
```

¿Qué hace este comando?

Descarga el repositorio remoto y toda su historia a tu máquina mediante SSH.

2.2 El Ciclo de Trabajo en Windows: Working Tree, Index y Commit

```
POWERSHELL

git status
git diff
git diff --staged
```

¿Qué hace este comando?

Consulta el estado de los archivos y compara diferencias antes y después de enviarlas al área de preparación (staging).

2.3 Staging Selectivo y Estándar Conventional Commits

```
git add -p .\Controlador.cs
```

¿Qué hace este comando?

Permite preparar de forma interactiva fragmentos específicos de código.

```
git commit -m "feat(auth): agregar soporte para Azure AD"
```

¿Qué hace este comando?

Confirma los cambios usando el estándar de commits convencionales (`feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`).

2.4 Sincronización con GitHub

```
git fetch origin  
git pull --rebase origin main  
git push -u origin main
```

¿Qué hace este comando?

Descarga novedades de GitHub, sitúa tus commits locales encima de los remotos y publica tu rama con seguimiento permanente.

2.5 Control de Archivos Ignorados en Windows

POWERSHELL

```
$gitignore = @"
Thumbs.db
desktop.ini
$RECYCLE.BIN/
.vs/
bin/
obj/
*.user
*.suo
*.log
"@
Set-Content -Path ".gitignore" -Value $gitignore
git rm -r --cached .\bin
```

¿Qué hace este comando?

Genera el archivo de exclusión de artefactos de Windows y Visual Studio y remueve carpetas rastreadas accidentalmente.

Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)

3.1 Gestión de Ramas con PowerShell y Posh-Git

```
git switch -c feature/conexion-sql
git push -u origin feature/conexion-sql
git switch main
git branch -d feature/conexion-sql
git push origin --delete feature/conexion-sql
```

¿Qué hace este comando?

Crea una nueva rama, la publica en GitHub, regresa a `main`, borra la rama local y la elimina en GitHub.

3.2 Los Tres Métodos de Merge en GitHub

- **Merge Commit:** Fusión con commit conmemorativo que conserva todo el historial de ramas.
- **Squash and Merge:** Aplasta todos los commits de la rama en un único commit conciso sobre la rama principal.
- **Rebase and Merge:** Coloca los commits individualmente al final de la rama destino, logrando una historia completamente lineal.

3.3 Modelos de Flujo: GitHub Flow, Git Flow y Forking

```
gh repo fork microsoft/PowerShell --clone
Set-Location .\PowerShell
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

¿Qué hace este comando?

Crea un fork de un repositorio de código abierto, lo clona a tu equipo y mantiene sincronizada tu rama `main` con el proyecto original.

3.4 Operaciones Avanzadas: Stash, Cherry-Pick y Rebase Interactivo

POWERSHELL

```
git stash save "Trabajo preliminar sin terminar"
git stash pop
git cherry-pick 4f2a1b9
git rebase -i HEAD~4
```

¿Qué hace este comando?

Guarda cambios en el búfer temporal, los restaura, aplica un commit de otra rama o reescribe interactivamente los últimos 4 commits.

3.5 El Salvavidas: Recuperación con Reflog

POWERSHELL

```
git reflog
git reset --hard "HEAD@{2}"
```

¿Qué hace este comando?

Consulta el historial cronológico de movimientos de `HEAD` en Windows y revierte cualquier error destructivo al estado de hace dos operaciones.

Parte IV: Soluciones por Temas a la Edición Concurrente del Mismo Archivo

4.1 Tema 1: Prevención y Buenas Prácticas de Equipo

1. **Trabajar en ramas independientes:** Prohibir commits directos sobre `main`.
 2. **Modularización:** Separar archivos extensos en clases y componentes pequeños.
 3. **Draft Pull Requests:** Publicar PRs en borrador para que el equipo visualice qué archivos están bajo edición.
 4. **Descargas frecuentes:** Ejecutar `git pull --rebase origin main` periódicamente.
-

4.2 Tema 2: Fusión Automática en Distintas Líneas

A screenshot of a Windows PowerShell terminal window. The window has a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar on the right says "POWERSHELL". The terminal shows two lines of text: "git pull --rebase origin main" and "git push origin main".

```
git pull --rebase origin main
git push origin main
```

¿Qué hace este comando?

Trae los cambios del compañero y coloca tus commits encima. Git resuelve la fusión automáticamente si las líneas no se solapan.

4.3 Tema 3: Conflicto Directo de Fusión con VS Code

```
git config --global diff.tool vscode
git config --global difftool.vscode.cmd 'code --wait --diff $LOCAL $REMOTE'
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd 'code --wait $MERGED'

git mergetool
git add .\Controlador.cs
git rebase --continue
git push origin main
```

¿Qué hace este comando?

Configura e inicia Visual Studio Code como herramienta de fusión gráfica de 3 vías, marca el archivo resuelto y concluye el rebase.

```
git rebase --abort
```

¿Qué hace este comando?

Cancela el rebase y regresa al estado limpio previo al conflicto.

4.4 Tema 4: Elección Total de Versión

```
# Conservar mi versión completa:
git checkout --ours .\Controlador.cs
git add .\Controlador.cs && git commit -m "resolve: mantener versión local"

# Aceptar la versión del compañero en su totalidad:
git checkout --theirs .\Controlador.cs
git add .\Controlador.cs && git commit -m "resolve: aceptar versión remota"
```

¿Qué hace este comando?

Fuerza la selección completa de una de las dos versiones sin editar línea por línea.

4.5 Tema 5: Cambios Locales sin Confirmar al hacer Pull

```
git stash save "Cambios locales en curso"
git pull --rebase origin main
git stash pop
```

¿Qué hace este comando?

Aparta tus modificaciones no guardadas, descarga las novedades de GitHub y reaplica tus cambios.

4.6 Tema 6: Push Rechazado por Desfase y Rebase Seguro

```
# NUNCA uses push --force
git pull --rebase origin main
git push origin main
```

¿Qué hace este comando?

Sitúa tus commits por encima de los que tu compañero subió a GitHub y los publica sin sobrescribir nada.

4.7 Tema 7: Resolución de Conflictos en Pull Requests

```
gh pr checkout 18
git merge origin/main
code .\Controlador.cs
git add .\Controlador.cs
git commit -m "merge: resolver conflictos con rama main"
git push origin HEAD
```

¿Qué hace este comando?

Descarga la rama del PR, incorpora `main`, resuelve conflictos en VS Code y actualiza el PR en GitHub.

4.8 Tema 8: Conflicto de Modificación vs Eliminación

```
POWERSHELL

# Para mantener el archivo modificado:
git add .\Controlador.cs && git commit -m "resolve: mantener archivo"

# Para aceptar el borrado:
git rm .\Controlador.cs && git commit -m "resolve: confirmar borrado"
```

¿Qué hace este comando?

Resuelve colisiones donde un usuario modificó el archivo y otro lo eliminó.

4.9 Tema 9: Archivos Binarios y Bloqueo con Git LFS en Windows

```
POWERSHELL

git lfs install
git lfs track "*.psd" --lockable
git lfs lock assets/disenio.psd
git lfs locks
git lfs unlock assets/disenio.psd
```

¿Qué hace este comando?

Configura Git LFS en Windows y bloquea archivos binarios en GitHub para prevenir modificaciones simultáneas.

Parte V: Herramientas Modernas de Productividad Avanzada

5.1 Git Worktrees en Windows: Trabajar en Múltiples Ramas sin Conmutar

```
POWERSHELL

# Extraer una rama en una carpeta paralela sin conmutar tu espacio actual
git worktree add ..\hotfix-windows hotfix/arreglo-urgente
Set-Location ..\hotfix-windows

git commit -am "fix: solucionar excepción crítica en Windows Server"
git push origin hotfix/arreglo-urgente

Set-Location ..\mi-proyecto-windows
git worktree list
git worktree remove ..\hotfix-windows
```

¿Qué hace este comando?

Crea un directorio independiente para trabajar simultáneamente en otra rama sin necesidad de hacer stash ni interrumpir tu tarea activa.

5.2 Depuración Binaria con Git Bisect y Auditoría con Blame

```
POWERSHELL

git bisect start
git bisect bad
git bisect good v1.2.0

# Tras probar cada compilación en PowerShell:
git bisect good # 0: git bisect bad

git bisect reset
```

¿Qué hace este comando?

Realiza una búsqueda binaria para encontrar automáticamente qué commit introdujo una regresión.

POWERSHELL

```
git blame -L 15,30 .\Servicio.cs
```

¿Qué hace este comando?

Muestra quién modificó cada línea del archivo, con el autor, fecha y hash de commit.

5.3 GitHub Codespaces desde PowerShell

POWERSHELL

```
gh codespace create --repo usuario/mi-proyecto --branch main
gh codespace list
gh codespace code -c nombre-del-codespace
```

¿Qué hace este comando?

Despliega un entorno de desarrollo completo en la nube de GitHub y lo abre en Visual Studio Code.

5.4 GitHub Copilot CLI en PowerShell

POWERSHELL

```
gh extension install github/gh-copilot
gh copilot suggest "listar procesos que usan mas de 500MB de RAM en PowerShell"
gh copilot explain "Get-Process | Where-Object WorkingSet -gt 500MB | Sort-Object WorkingSet
-Descending"
```

¿Qué hace este comando?

Agrega asistencia de IA generativa dentro de PowerShell para redactar o interpretar comandos complejos.

5.5 Git Hooks y Validación con Pre-commit en Windows

```
winget install Python.Python.3.12 -e
pip install pre-commit

$precommitConfig = @"
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: check-added-large-files
        args: ['--maxkb=5000']
      - id: detect-private-key
      - id: end-of-file-fixer
"@
Set-Content -Path ".pre-commit-config.yaml" -Value $precommitConfig
pre-commit install
```

¿Qué hace este comando?

Instala un gancho que valida automáticamente en cada commit local que no se suban archivos mayores a 5 MB ni claves privadas.

Parte VI: Gestión de Proyectos y Ecosistema GitHub

6.1 GitHub Issues, Milestones y Labels desde PowerShell

```
gh issue create --title "Excepción en conexión SQL" --body "Error de timeout bajo Windows Server 2022." --label "bug,windows"
gh issue list --assignee "@me"
gh issue close 14 --comment "Solucionado con el nuevo pool de conexiones."
```

¿Qué hace este comando?

Crea, audita y cierra incidencias formales vinculadas al repositorio desde PowerShell.

6.2 Pull Requests y Revisiones desde GitHub CLI

```
gh pr create --title "feat: módulo de compresión ZIP" --body "Implementación nativa con System.IO.Compression" --assignee "@me"
gh pr checkout 8
gh pr review 8 --approve -b "Validado en Windows 11."
```

¿Qué hace este comando?

Crea un Pull Request formal, descarga la rama de un compañero y emite una aprobación.

6.3 GitHub Projects (v2): Automatizaciones y Tableros

```
gh project list
gh project item-add 5 --owner "mi-empresa" --url "https://github.com/usuario/repo/issues/14"
```

¿Qué hace este comando?

Vincula una tarea al tablero Kanban de la organización.

6.4 GitHub Discussions y Wikis en Windows

POWERSHELL

```
git clone git@github.com:usuario/mi-proyecto-windows.wiki.git
Set-Location .\mi-proyecto-windows.wiki
Add-Content -Path "Guia-PowerShell.md" -Value "## Configuración recomendada"
git add Guia-PowerShell.md
git commit -m "docs: agregar guía de PowerShell a la wiki"
git push origin master
```

¿Qué hace este comando?

Clona, edita y sincroniza la documentación Wiki del proyecto como repositorio Git.

Parte VII: Automatización y CI/CD con GitHub Actions

7.1 Estructura de Pipelines y Sintaxis YAML

Los workflows se alojan en `.github/workflows/*.yaml` y automatizan pruebas y despliegues.

7.2 Workflows con Runners `windows-latest` y Shell `powershell`

Crea `.github/workflows/windows-ci.yaml` :

```
name: Windows .NET & PowerShell CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: windows-latest
    defaults:
      run:
        shell: pwsh

    steps:
      - name: Clonar código
        uses: actions/checkout@v4

      - name: Configurar SDK de .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '8.0.x'

      - name: Restaurar dependencias
        run: dotnet restore

      - name: Compilar solución en modo Release
        run: dotnet build --configuration Release --no-restore

      - name: Ejecutar pruebas unitarias
        run: dotnet test --no-build --verbosity normal --logger
"trx;LogFileName=test_results.trx"

      - name: Empaquetar artefactos de compilación
        run: |
          Compress-Archive -Path .\bin\Release\net8.0\* -DestinationPath .\App-Windows-
x64.zip

      - name: Subir artefacto compilado
        uses: actions/upload-artifact@v4
```

```
with:
  name: windows-application-binary
  path: .\App-Windows-x64.zip
```

7.3 Secretos, Matrices Multiplataforma y Caching

```
strategy:
  matrix:
    os: [windows-latest, ubuntu-latest]
    dotnet: ['7.0.x', '8.0.x']
```

7.4 Configuración de un Self-Hosted Runner como Servicio de Windows

```
New-Item -ItemType Directory -Path "C:\actions-runner" -Force
Set-Location "C:\actions-runner"
Invoke-WebRequest -Uri
"https://github.com/actions/runner/releases/download/v2.316.0/actions-runner-win-x64-
2.316.0.zip" -OutFile "runner.zip"
Expand-Archive -Path "runner.zip" -DestinationPath "."
.\config.cmd --url https://github.com/usuario/repo --token TU_TOKEN_AQUI
.\svc.cmd install
.\svc.cmd start
Get-Service "actions.runner.*"
```

¿Qué hace este comando?

Registra e instala un runner oficial privado como Servicio de Windows de ejecución continua.

Parte VIII: Distribución, Paquetes y Publicación

8.1 GitHub Releases: Binarios .exe, .msi y Archivos .zip



```
git tag -a v2.0.0 -m "release: versión 2.0.0 estable"
git push origin v2.0.0
gh release create v2.0.0 .\Instalador.msi .\App.zip --title "Versión 2.0.0 para Windows" --generate-notes
```

¿Qué hace este comando?

Publica formalmente la versión v2.0.0 en GitHub, genera notas automáticas y adjunta los instaladores.

8.2 GitHub Packages: Registro de Paquetes NuGet y npm



```
dotnet nuget add source "https://nuget.pkg.github.com/usuario/index.json" `
  -n "GitHubPackages" -u "TU_USUARIO" -p "TU_TOKEN_PAT" --store-password-in-clear-text
dotnet nuget push .\bin\Release\MiLibreria.1.0.0.nupkg --source "GitHubPackages"
```

¿Qué hace este comando?

Publica un paquete NuGet en el registro de paquetes privado de GitHub.

8.3 GitHub Pages: Publicación Automatizada

Alojamiento estático gratuito para documentación desplegado mediante GitHub Actions.

Parte IX: Seguridad, Gobernanza y Administración

9.1 Reglas de Protección de Ramas y Rulesets

Desde **Settings** → **Rules** → **Rulesets**:

- Prohibir commits directos a `main`.
- Exigir que los pipelines de Windows finalicen con éxito.
- Requerir revisiones aprobadas obligatorias.

9.2 Dependabot, Secret Scanning y Push Protection

```
version: 2
updates:
  - package-ecosystem: "nuget"
    directory: "/"
    schedule:
      interval: "weekly"
YAML
```

9.3 Análisis de Seguridad SAST con CodeQL

Análisis estático que detecta vulnerabilidades antes de fusionar PRs a producción.

9.4 Gobernanza con CODEOWNERS y Permisos

```
# .github/CODEOWNERS
* @mi-organizacion/tech-leads
*.ps1 @mi-organizacion/windows-admins
/.github/workflows/ @mi-organizacion/devops-team
PLAINTEXT
```

Parte X: Catálogo Maestro de Incidentes y Soluciones en Windows

Situaciones críticas de producción y su remediación en Windows:

10.1 Incidente 1: Fuga Accidental de Credenciales o Tokens

Escenario: Subiste un archivo `appsettings.json` o script de PowerShell con credenciales a GitHub.

1. **Revoca la credencial de inmediato** en el portal del proveedor (Azure, AWS, base de datos).
2. **Eliminar el archivo de todo el historial de Git en Windows:**

```
POWERSHELL
winget install Python.Python.3.12 -e
pip install git-filter-repo
git filter-repo --path appsettings.Production.json --invert-paths --force
git push origin --force --all
git push origin --force --tags
```

¿Qué hace este comando?

Pinta de nuevo toda la historia de commits eliminando físicamente cualquier rastro del archivo sensible.

10.2 Incidente 2: Push Rechazado por Archivo Mayor a 100 MB

Escenario: GitHub rechaza el push con el error `exceeds GitHub's file size limit of 100.00 MB`.

```
POWERSHELL
# Si el archivo está en el commit más reciente:
git reset --soft HEAD~1
git rm --cached .\archivo-pesado.zip
Add-Content -Path ".gitignore" -Value "`narchivo-pesado.zip"
git commit -m "chore: reconstruir commit sin archivo mayor a 100MB"
git push origin main
```

```
# Si el archivo quedó en commits intermedios antiguos:  
git filter-repo --strip-blobs-bigger-than 100M --force  
git push origin main
```

10.3 Incidente 3: Reversión Limpia de un Merge Roto en Producción

```
# 1. Localizar el hash del merge  
git log --oneline -n 5  
  
# 2. Revertir el merge seleccionando la rama padre 1 (-m 1)  
git revert -m 1 HASH_DEL_MERGE -m "revert: revertir merge que provocó fallo en producción"  
git push origin main
```

10.4 Incidente 4: Rebase de una Rama Compartida

Si un compañero reescribió una rama pública con rebase forzado:

```
git fetch origin  
git switch rama-afectada  
git rebase --onto origin/rama-afectada @{upstream}
```

¿Qué hace este comando?

Identifica los commits propios no incluidos en el rebase del compañero y los coloca sobre la nueva base sin duplicados.

10.5 Incidente 5: Restaurar una Rama Remota Borrada en GitHub

```
POWERSHELL

# Buscar el último commit de la rama en el reflog
git reflog | Select-String "rama-borrada"
# Supongamos que el hash es 4b8c2d1

# Recrear la rama y subirla
git switch -c rama-borrada 4b8c2d1
git push -u origin rama-borrada
```

10.6 Incidente 6: Corrección Masiva de Correo en Commits Históricos

```
POWERSHELL

git filter-repo --email-callback '
return email.replace(b"correo_erroneo@empresa.com", b"correo_correcto@empresa.com")
' --force
git push origin --force --all
```

10.7 Incidente 7: Ataques de Pwn Request en GitHub Actions

- Configura en **Settings** → **Actions** → **General**: **Require approval for all outside collaborators**.
- Nunca ejecutes código de PRs externos con `pull_request_target` si el pipeline utiliza secretos de Azure o producción.

10.8 Incidente 8: Bucle Infinito de Workflows en GitHub Actions

Incluye `[skip ci]` en los mensajes de commit generados por bots:

```
- name: Commit automático de versión
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "github-actions[bot]@users.noreply.github.com"
    git add .
    git commit -m "chore: incremento automático de versión [skip ci]"
    git push
```

10.9 Incidente 9: Optimización de Repositorios Pesados en Windows

```
git count-objects -vH
git reflog expire --expire=now --all
git gc --prune=now --aggressive
```

¿Qué hace este comando?

Limpia referencias caducadas y comprime al máximo la base de datos de objetos `.git` en disco.

10.10 Incidente 10: Tags o Etiquetas Desincronizadas en Windows

```
git tag -l | ForEach-Object { git tag -d $_ }
git fetch --tags --prune origin
```

¿Qué hace este comando?

Borra la lista local de tags desactualizados y descarga la lista oficial limpia desde GitHub.